

cg2-cg4-phase-a-summary

CodeGraph Phase A — captured evidence summary (CG2-CG4)

Field	Value
Revision	1
Created	2026-05-20
Last modified	2026-05-20
Status	active

Table of contents

- [CG2 — scaffold](#)
- [CG3 — install.sh](#)
- [CG4 — initialize + scan](#)
- [Anti-bluff verdict](#)

CG2 — scaffold

Created `tools/codegraph/`: `README.md`, `codegraph.version`, `install.sh`, `verify.sh` (honest non-zero stub — Phase C / CG10 fills it), `agents/` (with `.gitkeep`). Added CONST-053 `.gitignore` rules at the repo root for `tools/codegraph/`, `node_modules/`, `tools/codegraph/package*.json`, and `.codegraph/` (build/generated artefacts — never versioned).

All shell scripts: honest shebang (`#!/usr/bin/env bash`), `sh -n + bash -n clean`, in-source doc block (CONST-§11.4.18 / CONST-068).

CG3 — install.sh

`install.sh` is idempotent: reads the pinned version from `codegraph.version`, prechecks Node.js (`>=18 <25`) + npm, runs `npm install --prefix tools/codegraph`, resolves the binary to an absolute path, runs a `codegraph --version` self-check.

Version pin correction. The plan pinned `0.7.11`, but that version does not exist on npm (npm error code `ETARGET` — No matching version found for `@colbymchenry/codegraph@0.7.11`). See `cg3-install-attempt1-fail.txt`. npm view reports the available set ends at `0.7.12`; `0.7.11` was never published / was unpublished. Per incorporation-plan §8.4 (version drift is an expected, reviewed change) the pin was bumped to `0.7.12` — the current latest, still inside the supported Node window (`engines: node >=18 <25`).

Successful install (`cg3-install-success.txt`):

```
==> npm install --prefix ../tools/codegraph @colbymchenry/codegraph@0.7.1
added 51 packages in 6s
==> Resolved binary : ../tools/codegraph/node_modules/.bin/codegraph
==> Binary self-check: codegraph --version
0.7.12
==> CodeGraph 0.7.12 installed OK.
```

CG4 — initialize + scan

Per incorporation-plan §3.4, BOTH the meta-repo root and the inner `helix_code/` Go module were initialized (`codegraph init`) and scanned (`codegraph index`). `config.json` exclude lists were tuned to skip vendored / reference trees (`cli_agents/`, `cli_agents_resources/`, `Example_Projects/`, `dependencies/`, `github_pages_website/`, `external/`, `.codegraph/`).

Meta-repo root graph (`cg4-status-root.json`)

Metric	Count
files	39,022
nodes	624,092
edges	1,644,454
backend	native

Languages: c, cpp, csharp, go, java, javascript, jsx, kotlin, python, ruby, rust, scala, swift, tsx, typescript. Node kinds include 117,940 functions, 132,834 methods, 14,905 interfaces, 15,698 structs, 1,137 routes.

Inner Go module graph (`cg4-status-inner-helix_code.json`)

Metric	Count
files	1,541
nodes	32,959
edges	105,037
backend	native

The inner module is the real HelixCode Go domain code: 10,849 Go functions, 7,397 methods, 194 interfaces, 2,328 structs, 121 routes.

Sample query (`cg4-query-provider-root.json`)

`codegraph query Provider` against the root graph returns real HelixCode Go symbols — the `Provider` interface in multiple files under the actual repo tree (e.g. `helix_agent/MCP/submodules/.../provider.go`).

Anti-bluff verdict

PASS. CodeGraph genuinely installed (0.7.12, 51 packages) AND scanned the real HelixCode repository, producing two non-empty graphs:

- root: 624,092 nodes / 1,644,454 edges over 39,022 files;
- inner helix_code/: 32,959 nodes / 105,037 edges over 1,541 files.

No zero counts. `codegraph query` returns real symbols. This satisfies CONST-035 / Article XI §11.9 — the scan produced a genuine code-graph, not an empty / metadata-only artefact.